ToyDB: page buffering summary

Part 1 implementation and handoff | 2 November 2026

This document summarizes the completed ten-step PF page-buffering work. It records implementation details, validation, experiment results and the agreed design choice. Part 2 (slotted pages) is the next task for the teammate; part 3 (index construction) remains pending.

1. Compile and test the supplied code

Rebuilt the original PF source with GCC 13.3 using make -B tests. Both supplied programs, testhash and testpf, completed successfully. No source compatibility fixes were needed for that baseline. Legacy C warnings remain; passing tests do not imply the code has been fully modernized. Tests run in scratch directories to preserve the supplied files.

2. Understand the existing page flow

Open a file and obtain a PF descriptor. Request a page using its descriptor and page number. The buffer manager checks its hash table; a hit reuses RAM, while a miss obtains a frame and reads the page. The page remains fixed while in use. Unfixing releases it for possible replacement but leaves it cached. Dirty victims are saved before reuse, and closing a file flushes its dirty pages after all pages are unfixed.

The disk file contains a file header and fixed-size stored pages. RAM contains the open-file table, buffer frames and lookup structures. A PF descriptor indexes the open-file table; the entry contains the underlying OS descriptor. A buffered page is identified by the pair (PF descriptor, page number).

3. Make the buffer size configurable

Added PF_InitWithBufferSize(count), which validates the requested positive count, rejects initialization while files are open or after frames have been allocated, sets the capacity and calls the existing PF_Init. buf.c now checks a runtime capacity instead of the fixed limit. Frames are allocated on demand. Legacy callers retain the default of 20 frames; live resizing is unsupported.

4. Add replacement-policy choice when opening a file

Added PF_OpenFileWithPolicy(name, policy), with PF_POLICY_LRU and PF_POLICY_MRU. Each open-file entry stores its selected policy. PF_GetFilePolicy(fd) retrieves it. Invalid policies are rejected before opening the file. Existing PF_OpenFile(name) selects LRU, preserving existing callers. The pool remains shared across files.

Implementation: replacement, updates and counters

5. Implement MRU and verify LRU

PFbufInternalAlloc now receives the requesting file descriptor and retrieves its selected policy. LRU searches from the least-recent end of the global list; MRU searches from the most-recent end. Both skip fixed pages. If every frame is fixed, the request returns PFE_NOBUF. Dirty victims are saved using their own file descriptor, even when another file triggered eviction.

Recency follows the supplied implementation: new frames enter at the head; unfixing a page or calling PFbufUsed moves it to the head. Fixed pages cannot be victims regardless of list position.

6. Add an explicit operation to mark a page dirty

Added PF_MarkDirty(fd, pageNum), which validates the descriptor and page number and reuses PFbufUsed to mark a fixed page dirty and most recently used. It keeps the page fixed and does not immediately write it. A later PF_UnfixPage(..., FALSE) preserves dirty status. PF_UnfixPage(..., TRUE) remains supported. Dirty status is retained until successful writeback.

7. Add statistics

Added the pool-wide PF_Stats structure and PF_ResetStats(), PF_GetStats() and PF_PrintStats(). Resetting counters does not clear the buffer or flush it.

Measurement	Implemented meaning
Logical reads	Successful buffer fetches, including internal fetches and scans of free pages.
Logical writes	Successful update notifications through dirty unfixing or explicit dirty marking.
Physical reads / writes	Successful full-page transfers to/from the operating system; writes include eviction and final close flushes.
Buffer hits / misses	Successful fetches classified by whether the page was already cached.

Counts are operations, not distinct pages. File-header I/O, failed operations and fresh frame/page allocation itself are excluded. The OS may cache transfers, so physical I/O here does not mean hardware disk activity. A write query fetches a page and reports one update: one logical read plus one logical write. Do not report the same update through both dirty APIs unless two notifications are intended.

8. Check correctness with small examples

All 12 test cases passed again while preparing this handoff: the two original tests, teststats, five capacities (1, 2, 5, 20, 32), and LRU/MRU variants of testreplacement and testdirty. Tests verify exact victims, hit recency, cross-file policy selection, pinned-page protection, full-pool errors, dirty persistence through eviction/reopening, invalid inputs and exact counter totals/resetting.

Experiments and the agreed design decision

9. Run read/write experiments and produce graphs

Added benchmark.c and a configurable Python experiment runner. Defaults: 20 buffer frames, a 100-page file, 10,000 queries per run, sequential and random access, both policies, five read/write mixtures (100/0, 75/25, 50/50, 25/75, 0/100), and three repetitions: 60 trials total.

Each trial starts with an empty ToyDB pool after file creation and close. Setup is excluded. The same seeded page trace is used for both policies and all mixtures within a repetition. Writes are deterministically interspersed and toggle one byte; each reports dirty status once. Counts and monotonic elapsed time include the final dirty flush. CSV files retain each run and means; graph X-axes show read/write mixtures and Y-axes show page-operation statistics.

Default sequential result	LRU	MRU
Physical reads (each mixture)	10,000	8,100
Physical writes (50% writes)	5,000	4,050
Buffer hits	0	1,900

MRU reduces physical reads by 19% for this repeated-scan workload. Random-access results are similar between policies and vary by seed. These synthetic page workloads do not yet use student records and do not establish that MRU is always faster. OS caching and very short run times limit timing conclusions.

10. Document how to run it

Added the root README and PF-specific README, reproducible test runner, experiment configuration and graph runner. Experiment settings live in experiments.json so capacities, workloads, policies and repetitions can change without editing the C source. The exact run configuration is saved with the results. Build and execution commands are on the handoff page.

Design decision: global victims, file-specific policy

Question: The assignment says the replacement policy should be selectable per file, but does not specify whether replacement must be local to a file or global.

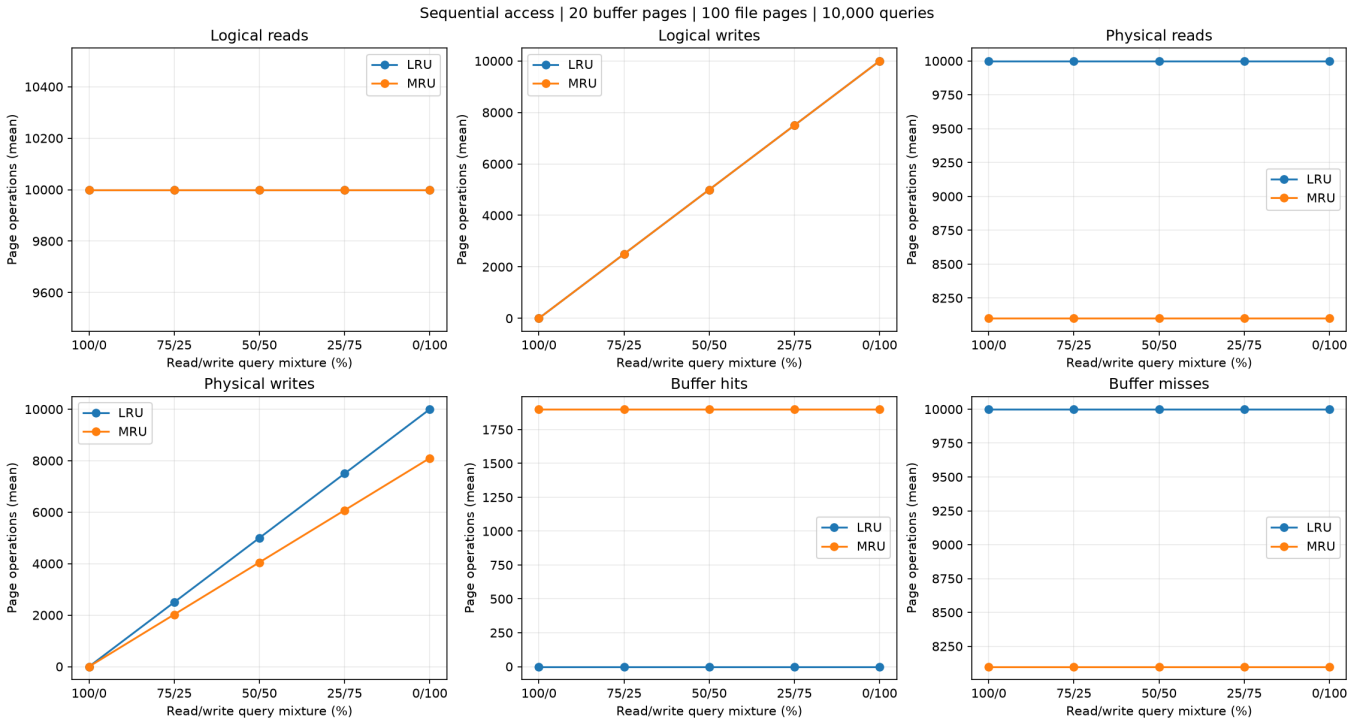
Choice 1 - local replacement: Evict only a page belonging to the requesting file. This requires allocation rules, such as how many frames each file receives. Fixed per-file allocations can restrict how many files can actively use the pool. A rule is also needed when all allocated pages are fixed or a new file has no frame of its own to replace.

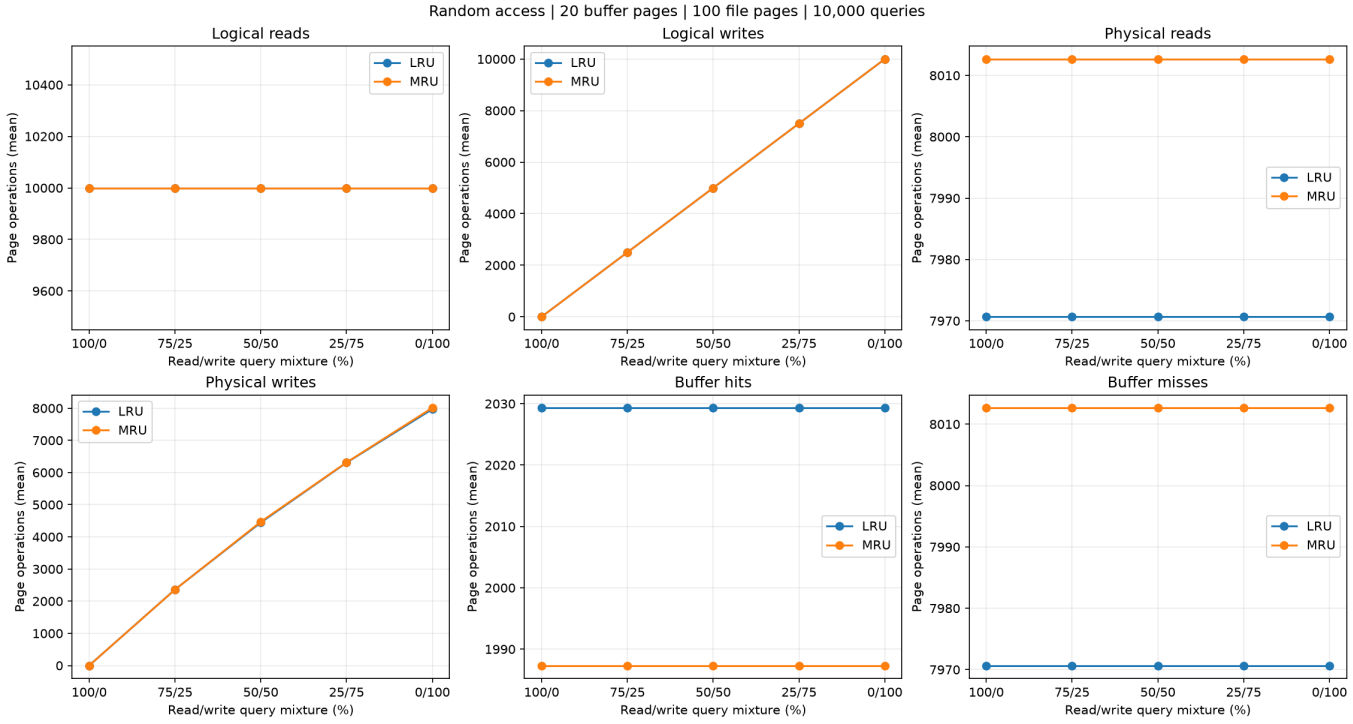
Choice 2 - global replacement: Use the requesting file's policy to evict any eligible global page. This is simpler and avoids per-file frame reservations.

Agreed decision: Use choice 2 for the first iteration. Complete all three assignment parts before revisiting local replacement. This is a documented interpretation of ambiguous requirements, not an explicit requirement in the assignment. The same decision is recorded in outputs/design-decisions/toydb-design-decisions.xlsx.

Experiment graphs

Each panel shows the mean of three repetitions. Lines overlap where both policies have identical counts. Configuration: 20 frames, 100 file pages, 10,000 queries per run.





Source outputs: outputs/page-buffering/results.csv, summary.csv and config.json. Elapsed times are available in the CSV files.

Teammate handoff: starting part 2

Build the slotted-page record layer on top of the PF interfaces. Store page headers, a slot directory and variable-length record bytes inside the PF-provided data area (PF_PAGE_SIZE = 4096 bytes). Do not alter the PF disk-page header. Decide the record identifier format (typically page number plus slot number) and coordinate it with the later roll-number index.

For insertion/deletion: fetch or allocate a page, modify its bytes, mark it dirty and unfix it on every completed path. For scanning: fetch each page, inspect its live slots and unfix it before moving on. Do not retain page pointers after unfixing. Avoid keeping many pages fixed; the global pool can otherwise run out of eligible frames.

```
/* Illustrative call sequence; check every return value. */
PF_InitWithBufferSize(20);
fd = PF_OpenFileWithPolicy("students.pf", PF_POLICY_LRU);
PF_GetThisPage(fd, pageNum, &pageBuf);
/* Read or update the slotted-page bytes in pageBuf. */
PF_MarkDirty(fd, pageNum);          /* only after updates */
PF_UnfixPage(fd, pageNum, FALSE);
PF_CloseFile(fd);
```

Part 2 still needs insertion, deletion, sequential scanning, student.txt loading, space-utilization metrics and a table comparing variable-length storage with fixed-length storage at different maximum record lengths. Part 3 still needs the index-construction comparison and sorted bulk loading. AM-layer compilation/tests have not been validated in this PF task.

Build, test and reproduce from the repository root

```
make -B -C toydb/pflayer tests benchmark
python3 toydb/pflayer/run_tests.py

# Python environment for graphs (matplotlib required):
python3 -m venv /tmp/toydb-venv
/tmp/toydb-venv/bin/pip install matplotlib
/tmp/toydb-venv/bin/python toydb/pflayer/run_experiments.py
```

Edit toydb/pflayer/experiments.json before running experiments. Optional runner arguments: --config path.json --output directory. The current machine also has matplotlib under /tmp/sailor-plot-deps; that temporary installation is not part of a Git checkout. Rebuild source after pulling; do not rely on supplied or regenerated binaries.

File map for review

Category	Files (relative to toydb/pflayer unless noted)
Modified implementation	pf.c, pf.h, pftypes.h, buf.c; Makefile builds all tests and benchmark.
Added tests and runner	testcapacity.c, testreplacement.c, testdirty.c, teststats.c, run_tests.py. Original testpf.c and testhash.c remain in use.
Added experiments	benchmark.c, experiments.json, run_experiments.py.
Documentation / outputs	Root README.md; PF README.md; outputs/page-buffering/ CSVs, config and graphs; decision spreadsheet; root summary.pdf.

Source basis: assignment Objective and submission instructions; supplied pf.ps and PF implementation; current README files, PF headers, test runner and saved experiment configuration/results. No commit or push was performed as part of preparing this summary.